

12

Garbage Collectors and Dynamic CDS Archives

This chapter includes 15 problems covering garbage collectors and **Application Class Data Sharing (AppCDS)**.

By the end of this chapter, you'll have a profound understanding of how a **garbage collector (GC)** works and how you can tune it for maximum performance. Moreover, you'll have a good understanding of how AppCDS can boost your application startup.

Problems

Use the following problems to test your advanced programming prowess in garbage collectors and application class data sharing in Java. I strongly encourage you to give each problem a try before you turn to the solutions and download the example programs:

- 243. **Hooking the garbage collector goal:** Introduce Java garbage collectors quickly. Highlight the main objectives (advantages) and disadvantages of a garbage collector.
- 244. **Handling the garbage collector stages:** List and briefly describe the most common stages of a garbage collector.
- 245. **Covering some garbage collector terminology:** A garbage collector has specific terminology. Provide here the main terms used in conjunction with garbage collectors.
- 246. **Tracing the generational GC process:** Exemplify and explain a hypothetical scenario containing several consecutive runs of a generational garbage collector.
- 247. **Choosing the correct garbage collector:** List and explain the three main factors that should be considered for choosing the correct garbage collector.
- 248. **Categorizing garbage collectors:** Highlight the main categories of garbage collector across JDK's evolution.
- 249. **Introducing G1:** Provide a brief introduction to the G1 GC, including its design principles.
- 250. **Tackling G1 throughput improvements:** List the main improvements of G1 GC throughput across JDK versions.
- 251. **Tackling G1 latency improvements:** List the main improvements of G1 GC latency across JDK versions.
- 252. **Tackling G1 footprint improvements:** List the main improvements of the G1 GC footprint across JDK versions.
- 253. **Introducing ZGC:** Provide a brief introduction to the Z Garbage Collector.

- 254. **Monitoring garbage collectors:** Explain and exemplify at least one tool for monitoring garbage collectors.
- 255. **Logging garbage collectors:** Provide the steps needed to log the garbage collector activity. Moreover, highlight some tools capable of analyzing and plotting the logged data.
- 256. **Tuning garbage collectors:** Explain how to tune garbage collectors, including G1 and ZGC.
- 257. **Introducing Application Class Data Sharing (AppCDS, or Java's Startup Booster):** Give a quick and practical guide to using CDS and AppCDS in JDK 10/11, 13, and 19.

The following sections describe solutions to the preceding problems. Remember that there usually isn't a single correct way to solve a particular problem. Also, remember that the explanations shown here include only the most interesting and important details needed to solve the problems. Download the example solutions to see additional details and experiment with the programs at

<https://github.com/PacktPublishing/Java-Coding-Problems-Second-Edition/tree/main/Chapter12>.

243. Hooking the garbage collector goal

Every programming language has to manage memory usage. Some programming languages delegate this task to program-

mers, while others leverage different mechanisms to partially control how memory is used. Java programmers can focus 100% on the functionalities of the application and let the *garbage collector* manage how memory is used.

The name *garbage collector* suggests an entity capable of finding and collecting garbage from memory. Actually, a garbage collector is a very complex process representing the climax of Java memory management that is capable of tracking every object from the heap and identifying and removing the ones that are not used/referenced by the application. The main advantages of a garbage collector include:

- The Java programmer doesn't need to manually handle the allocation/deallocation of memory.
- The Java programmer doesn't need to deal with *dangling* and *wild pointers* (https://en.wikipedia.org/wiki/Dangling_pointer).
- In a wide range of scenarios, a garbage collector prevents *memory leaks* (https://en.wikipedia.org/wiki/Memory_leak). However, this issue is not 100% covered.

While these advantages are major, there are a few disadvantages as well:

- A garbage collector itself is a resource that needs CPU power to work. We're talking about CPU power that is in addition to the CPU power needed by the application. More garbage collector activity requires more CPU power.
- The programmer cannot control the garbage collector scheduler. This may cause performance issues at peaks or when the application deals with intensive computations.
- Some garbage collectors cause long and unpredictable pauses of the application.
- Learning and tuning the correct garbage collector can be really cumbersome.

In the next problems, we'll go deeper into this topic.

244. Handling the garbage collector stages

During its work, GC passes through different stages or steps. It can pass through one or more of the following stages:

- *Mark* – In this stage, the GC identifies and marks (or paints) all pieces of memory (blocks) that are used (have references) and not used (have no references). The marked (painted) blocks are called *live objects*, while the rest are called *non-live objects*. *Imagine that you go to the pantry and*

identify all the fresh fruits and vegetables and separate them from the spoiled ones.

- *Sweep* – In this stage, the GC removes all *non-live objects* from memory. *Next, you take all the spoiled fruits and vegetables out of the pantry and throw them away.*
- *Compact* – In this stage, the GC attempts to group the *live objects* closer together – in other words, it arranges the live objects at the start of the heap in a continuous sequence of memory blocks. So, compacting involves *defragmentation* and *relocation* of the *live objects*. The goal of compaction is to obtain large memory blocks that are free and ready to serve other objects. *Next, we go to the pantry and stack all the fruits and vegetables in crates so that we get as much free space as possible. We will use this space for other fruits and vegetables that we are going to buy.*
- *Copy* – This is another stage dedicated to organizing memory. It is an alternative to the *mark* stage. In this stage, the GC moves the *live objects* into a so-called *ToSpace*. The rest of the objects are considered *non-live* and remain in the so-called *FromSpace*.

Typically, a GC follows one of these three scenarios:

- Mark -> Sweep -> Compact
- Copy
- Mark -> Compact

Next, let's cover some GC terminology.

245. Covering some garbage collector terminology

Garbage collection has its own terminology that it is essential to know in order to better understand how it works. Some of these terms are presented here; we start with *epoch*, *single pass*, and *multiple passes*.

Epoch

A GC works in cycles. A complete cycle of a GC is known as an *epoch*.

Single and multiple passes

A GC can handle its internal steps in a single pass (*single-pass*) or multiple passes (*multi-pass*). In the case of *single-pass*, the GC groups multiple steps and handles them in a single run. On the other hand, in the case of *multi-pass*, the GC handles multiple steps in a sequence of several passes.

Serial and parallel

A GC is considered *serial* if it uses a single thread. On the other hand, a GC is considered *parallel* if it uses multiple threads.

Stop-the-World (STW) and concurrent

A GC is of the type *Stop-the-World* (STW) if it has to stop (temporarily suspend) the application execution in order to carry out its cycle. On the other hand, a GC is *concurrent* if it is capable of running at the same time as the application without affecting its execution.

Live set

A GC *live set* represents all the *live objects* of the current application. If there is no memory leak (or other issues), then the *live set* should have a constant load factor and a relatively constant size. During application execution, objects are added/removed from the heap and from the *live set* respectively.

Allocation rate

Java allows us to set the size of the heap memory via the `-Xmx` options. This size should not exceed the memory available on your machine (server) and should be big enough to serve the *live set*. This can be achieved by taking into account the *alloca-*

tion rate, which is expressed as the amount of memory (for instance, MB) allocated per unit of time (for instance, seconds).



Important note

As a rule of thumb, try to set the heap size as 2.5 to 5 times the average size of the *live set*.

In other words, when many objects are created, there will be many cleanups as well. This means that the GC will run at a high frequency and will need a higher *allocation rate*.

NUMA

NUMA is the acronym for non-uniform memory access. A processor has its own memory (called local memory) but it can also access the memory of other processors. Access to its local memory is faster than access to non-local memory. Basically, NUMA is a memory architecture that attempts to optimize access to local memory.

Region-based

A *region-based* GC divides the heap into smaller (eventually equal) regions/chunks of memory (for instance, G1 and ZGC

are *region-based* GCs). Each such region can be allocated for different purposes.

Generational garbage collection

Generational garbage collection is an algorithm that excels in handling short-living objects. A GC that implements this algorithm is called a *generational GC*.

This algorithm distinguishes between *young* and *old* objects and keeps them separate. The *young* objects are kept in an area called the *Young* generation or *Nursery* space, while the *old* objects are kept in an area called the *Old* generation or *Tenured* space. The following figure highlights the transitions of objects through the *Young* and *Old* generations:

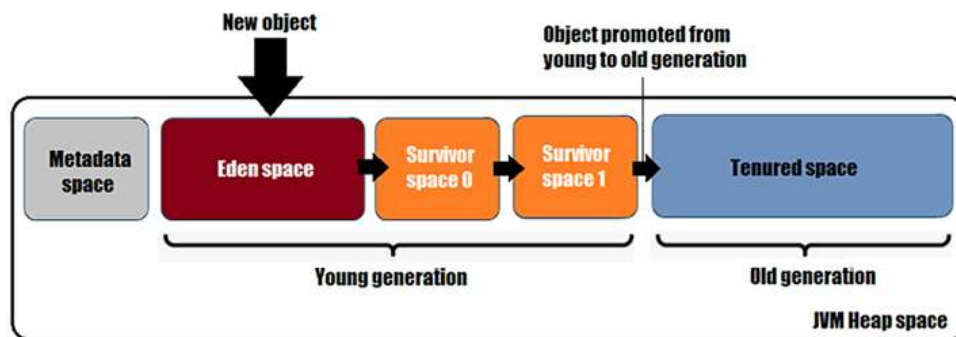


Figure 12.1: The transitions of objects through Young and Old generations

As you can see, the *Young* generation is divided into two regions/spaces, named the *Eden* region or *Eden* space and the *Survivor* region or *Survivor* space. Initially, the *Young* generation is empty.

Objects that are newly created are placed in the *Eden* space by default. However, there is an exception for extremely large objects, called *humongous objects*, that exceed 50% of the region's size. These objects are placed directly into the *Old* generation area, which may cause performance issues due to the increased occurrence of *major/full* GC events. It is important to note that GCs can trigger different types of events:

- *MinorGC* – This event occurs in the *Young* generation when the *Eden* space becomes full. Its purpose is to collect *non-live* objects and promote the remaining ones into the *Survivor* space. This event is the most commonly triggered by a GC.
- *MajorGC* – This event occurs in the *Old* generation and is responsible for collecting garbage from this area.
- *MixedGC* – This is a *MinorGC* event followed by reclaiming the *Old* generation.
- *FullGC* – Clean up the *Young* and *Old* generations and perform compacting of the *Old* generation (we can programmatically force a *FullGC* via `System.gc()` or `Runtime.getRuntime().gc()`).

Next, let's return to the topic of the *Young* generation. During the *epoch* (a GC complete cycle), the objects that survive (have not been garbage collected) are promoted into the *Survivor* space (the GC algorithm chooses between *Survivor space 0* (known as *S0* or *FromSpace*) or *1* (known as *S1* or *ToSpace*)). The objects that don't fit into the *Survivor* space (if any) will be moved into the *Tenured* space – this is known as *premature promotion*. Usually, the GC handles the *Eden* space pretty quickly via the *MinorGC* events. Using local variables with short-living methods encourages the usage of *Eden* space and sustains the GC's performance.

Objects that are considered old enough (they have survived during multiple epochs) are eventually promoted into the *Old* generation. This is an area typically (but not mandatorily) larger than the *Young* generation. Some GCs use a fixed delimitation between these areas (for instance, the **Concurrent Mark Sweep (CMS)** GC) while others use an elastic boundary between these areas (for instance, the G1 GC). The *Old* generation area takes a relatively long time to be garbage collected and has a lower frequency than the *Young* generation.

As you can see in *Figure 12.1*, the heap also contains an area named the *Metadata* space. Before JDK 8, this area was named *PermGenSpace* or *Permanent* generation. This area is used to store classes and methods. This area is specially designed to

grow beyond the heap size into the native memory (if the size of this area goes beyond the physical memory, the operating system will use virtual memory – but be aware that moving data between physical and virtual memory is a costly operation that will affect the application performance). Via *Metadata* space, JVM avoids out-of-memory errors. However, this area can be garbage collected in order to remove unused classes/methods. In this context, there are a few flags that can help us to tune it, but we will cover these flags in *Problem 256*.

246. Tracing the generational GC process

In this problem, let's start from an arbitrary initial state of a generational GC and follow a few hypothetical epochs (generally, all generational GC works more or less as you'll see in this problem). We start with the following diagram:

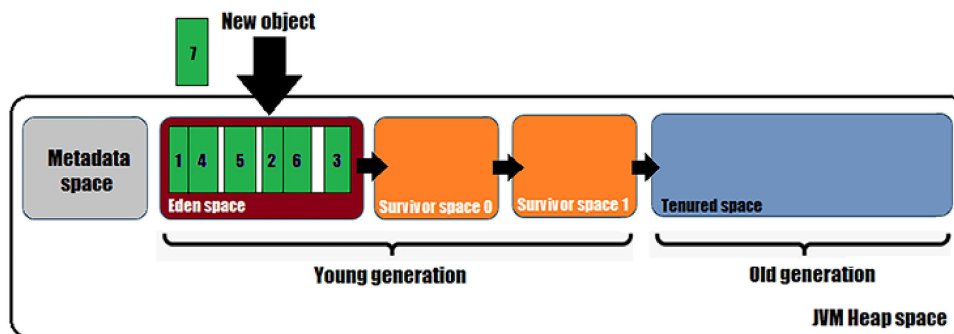


Figure 12.2: GC initial state

At its initial state, the GC has an almost full *Eden* space (it stores objects 1, 4, 5, 2, 6, and 3, and some free space – represented by those white gaps between objects) and empty *Survivor* and *Tenured* spaces. Moreover, object 7 should be added in the *Eden* space but there is not enough memory for it. When the *Eden* space cannot accommodate more objects, the GC triggers a *MinorGC* event. First, the *non-live objects* are identified. Here (as you can see in the following diagram), we have three objects (5, 2, and 3) that should be collected as garbage:

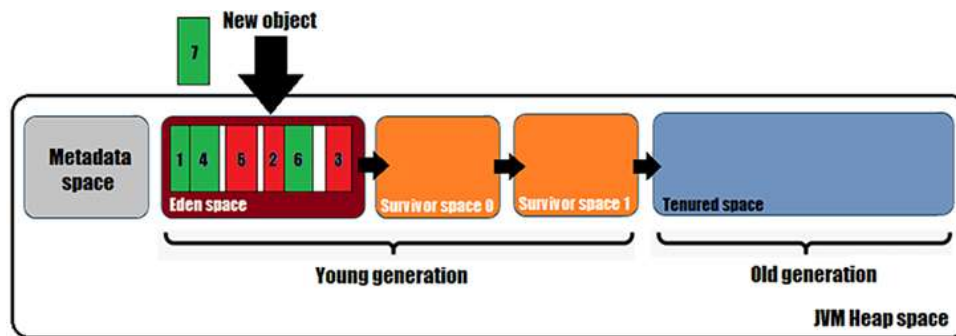


Figure 12.3: Identify the non-live objects from the Eden space

These three objects are collected as garbage, so they are removed from the heap. Next, the *live objects* (1, 4, and 6) are moved into *Survivor space 0*. Finally, the new object (7) is added into the *Eden* space, as in the following figure:

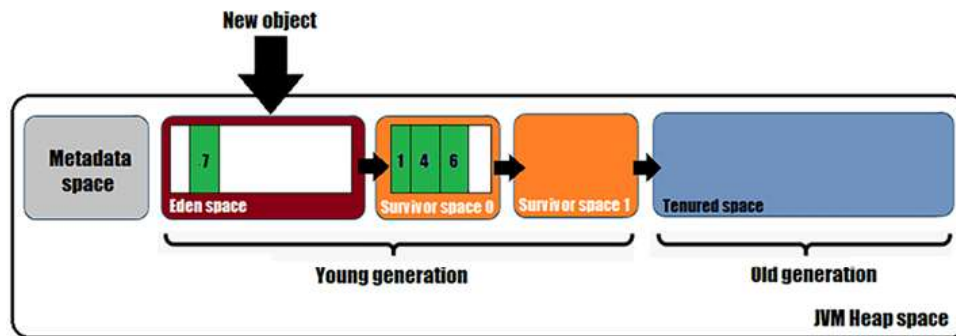


Figure 12.4: Removing objects from memory (5, 2, and 3), moving objects to Survivor space 0 (1, 4, and 6), and adding object 7 into the Eden space

Here, an *epoch* (complete GC cycle) has ended.

Later on, more objects are added into the *Eden* space until it is almost full again:

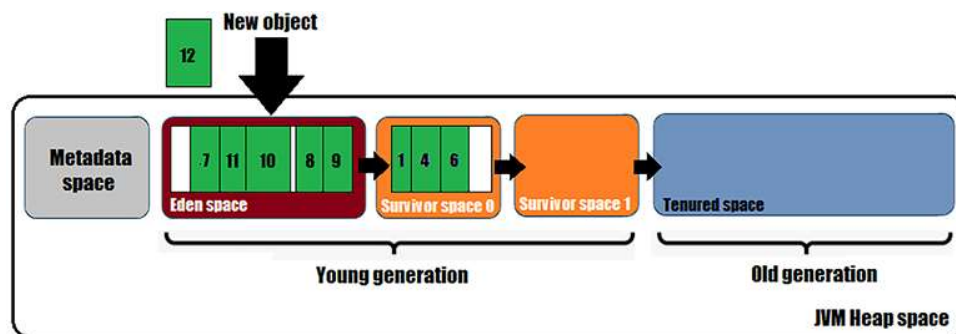


Figure 12.5: The Eden space is almost full again

Adding the new object (12) requires a *Minor GC* event. Again, the *non-living objects* are identified as follows:

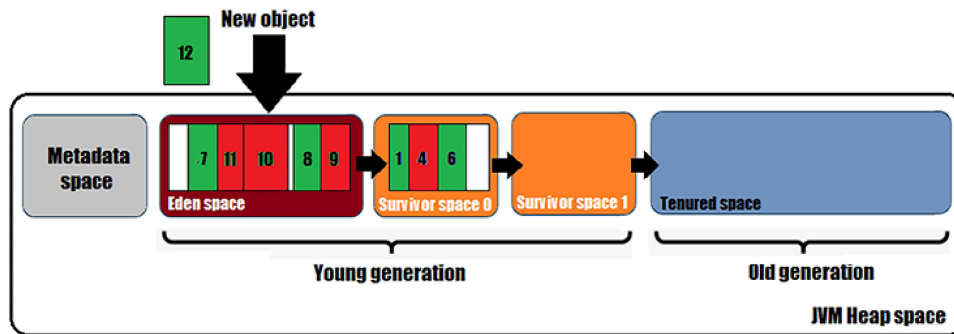


Figure 12.6: There are non-live objects in the Eden and Survivor 0 spaces

There are four objects that should be collected as garbage. In the *Eden* space, there are three objects (11, 10, and 9), and in *Survivor space 0*, there is one object (4). All four of these objects are removed from the heap. The *live objects* from *Survivor space 0* (1 and 6) are moved to *Survivor space 1*. The *live objects* from the *Eden* space (7 and 8) are also moved into *Survivor space 1*. At any moment in time, one of the *Survivor* spaces is empty. Finally, the new object (12) is added to *Eden* space, as in the following figure:

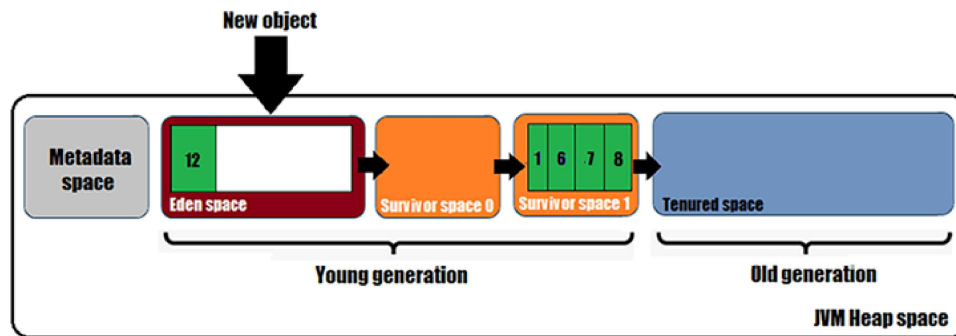


Figure 12.7: At the end of another epoch

Here, another epoch has ended.

Next, objects 13, 14, 15, and 16 are added to the *Eden* space, which is almost full again:

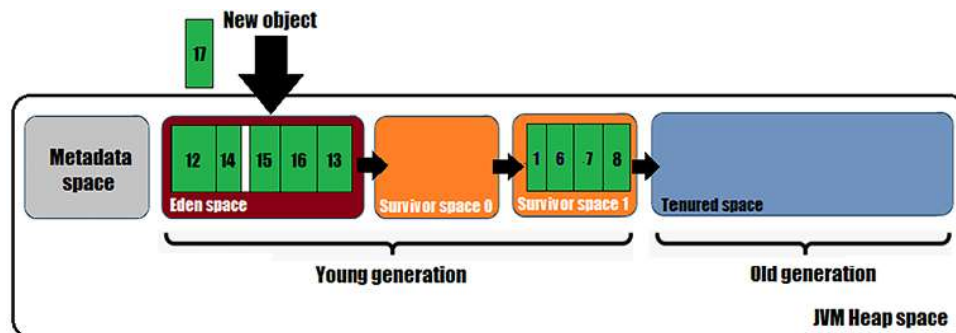


Figure 12.8: There is no memory available for object 17

Being almost full, the *Eden* space cannot accommodate the new object, 17. A new *Minor GC* event is triggered and objects 12,

15, 16, 13, 6, and 8 are identified as *non-live objects*:

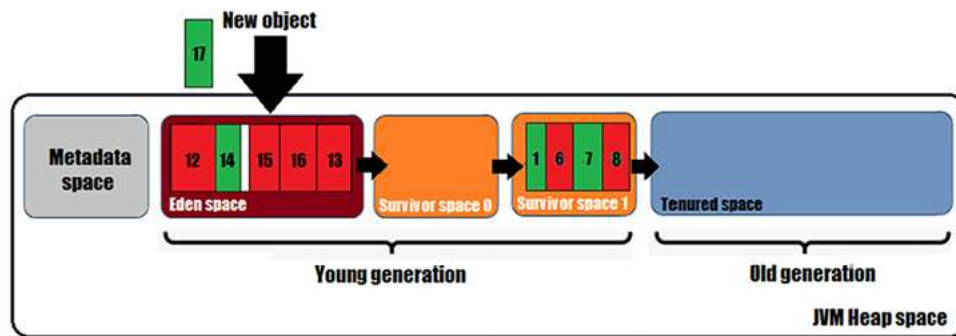


Figure 12.9: There are several non-live objects in different spaces

These objects (12, 15, 16, 13, 6, and 8) are removed from the heap. Next, object 14 is moved from the *Eden* space to *Survivor space 0*. Afterward, objects 1 and 7 (from *Survivor space 1*) are moved into *Survivor space 0*. Finally, the new object 17 is moved into the *Eden* space, as in the following figure:

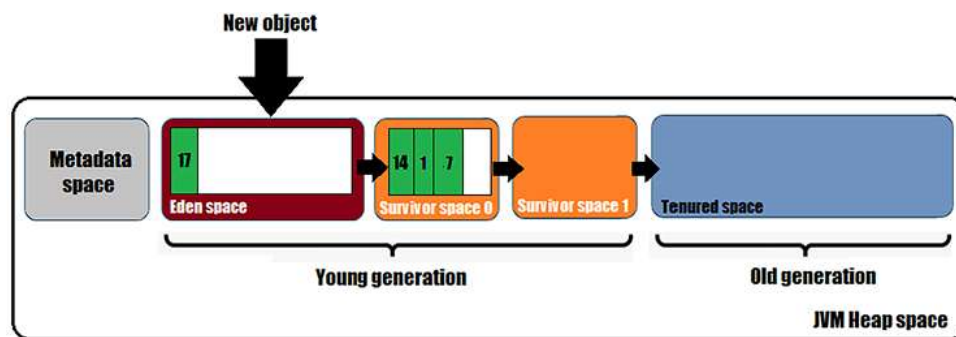


Figure 12.10: The new object (17) is added to the *Eden* space

Here, another *epoch* has ended.

We repeat the scenario and fill up the *Eden* space again. We stop when object 22 should be added into the *Eden* space:

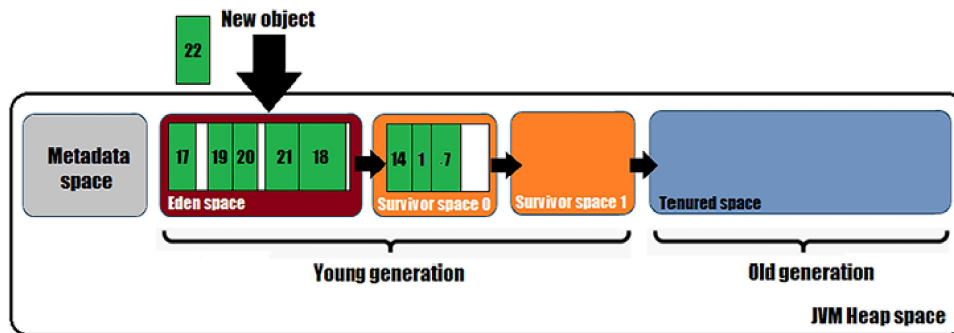


Figure 12.11: Trying to add in Eden space object 22

As we already know, the GC marks all the *non-live objects* (here, 17, 21, 18, and 7):

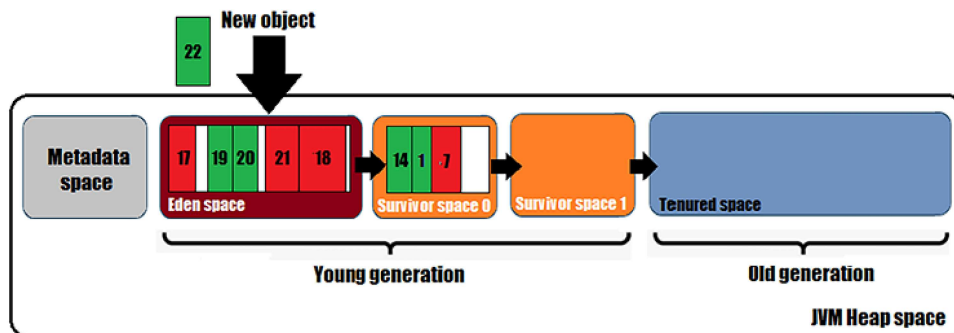


Figure 12.12: Marking the non-live objects

This time, the GC promotes object 1 (when it is considered old enough) from the *Young* generation to the *Old* generation. Next, the objects from the *Eden* space (19 and 20) and the objects from *Survivor space 0* (14) are moved into *Survivor space 1*. The result is sketched in the next figure:

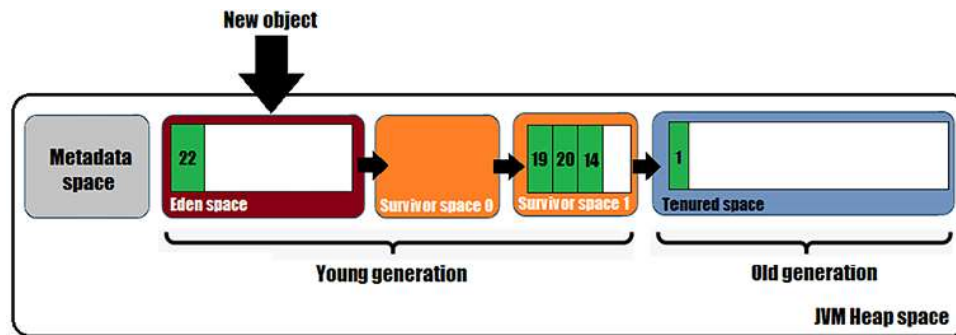


Figure 12.13: We have the first object promoted to Old generation

At the end of this *epoch*, we finally have an object (1) in *Tenured* space. Continuing to run *epoch* after *epoch* will eventually fill up the *Tenured* space, which will not be able to accommodate more objects. In other words, the *Minor GC* events (which are *stop-the-world* events) will reclaim the memory of the *Young* generation until the *Old* generation is full. When that happens, a *Mixed GC* or even *Full GC* event will be triggered (the *Full GC* is also an STW event and will handle the *Metadata* space as well).

In a nutshell, this is how a GC works. Of course, there are many other internal/external factors that may influence the GC's decisions.

247. Choosing the correct garbage collector

As you'll see in the next problem, Java allows us to choose between several garbage collectors. There is no silver bullet, so choosing the correct garbage collector for your particular application is an important decision that should be made based on three factors: *throughput*, *latency*, and *footprint*.

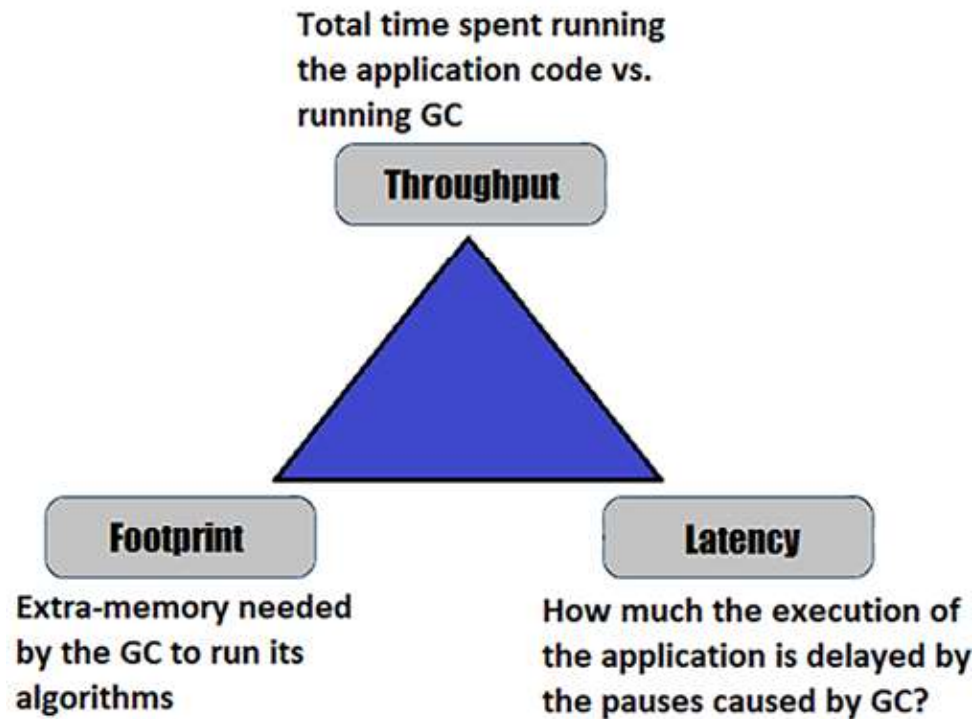


Figure 12.14: The factors that affect the choice of GC

Throughput represents the total time spent running the application code vs. running the GC. For instance, your application may run 97% of the total time, so you have a throughput of 97%. The remaining 3% is the time spent running the GC.

Latency measures how much the execution of the application is delayed by pauses caused by the GC. This is important because latency can affect the application's responsiveness.

These pauses may lead, at the interactivity level, to an unpleasant experience for the end users.

Footprint represents the extra memory needed by the GC to run its algorithms. This is the memory needed in addition to the memory used by the application itself.

Choosing the proper GC based on these three factors is a very subjective decision. You may need a massive throughput while you can bear latencies, or you may not be able to afford latencies because you have high interactivity with the end users, or your scalability is in direct correlation with limited physical memory, so you are really interested in the footprint factor. As you'll see in the next problem, each GC type has its own advantages and disadvantages in the context of these three factors.

248. Categorizing garbage collectors

Garbage collectors have evolved exactly as Java itself has evolved. Today (JDK 21), we distinguish between several GC types, as follows:

- Serial garbage collector
- Parallel garbage collector
- Garbage-First (G1) collector

- Z Garbage Collector (ZGC)
- Shenandoah Garbage Collector (not generational)
- Concurrent Mark Sweep (CMS) collector (deprecated)

Let's tackle the main aspects of each GC type.

Serial garbage collector

The serial garbage collector is an STW single-threaded generational collector. Before running its own algorithms, this GC freezes/pauses all the application threads. This means that this GC is not suitable for multi-threaded applications such as server-side components. However, being focused on a very small footprint (useful for small heaps), this collector is a good fit for single-threaded applications (and single-processor machines) that can easily accommodate and tolerate a significant latency (for instance, batch jobs or bulk processing).

Parallel garbage collector

The parallel garbage collector is an STW multi-threaded generational collector. Before running its own algorithms, this GC freezes/pauses all the application threads, but it speeds up the garbage collection by using multiple threads. In other words, this GC can take advantage of multi-processor machines and can be a good fit for multi-threaded applications that use

medium/large datasets. This GC is focused on throughput rather than latency and comes with pauses of 1 second or more. So, if you are in a multi-threaded context that can afford pauses of 1 second or more, then this GC is the right choice.

Garbage-First (G1) collector

The Garbage-First (G1) collector is an STW multi-threaded, region-based, generational collector focused on balanced performance. This GC was introduced in JDK 7 update 4 as a default (since JDK 9) solution that sustains high throughput and low latency (a few hundred milliseconds). The price to pay for this performance is a more frequent rate of epochs. The GC will run more often, so be prepared to provide a CPU ready to accommodate more cycles than other GCs. This GC was designed for server-style applications that are executed on multi-processor machines with massive memory (large heap size). Also known as a *mostly concurrent* collector, G1 performs heavily next to the application using equally sized spaces/regions (from 1 to 32 MB). So, if you can afford a large heap size and need low latency, then G1 is the proper choice. We will talk in detail about G1 in subsequent problems.

Z Garbage Collector (ZGC)

Z Garbage Collector (ZGC) was introduced for production starting with JDK 15 as a low-latency GC that can handle large heap sizes (terabytes). Like G1, ZGC works concurrently but it guarantees to not stop the application threads for more than a few milliseconds (the documentation even states that ZGC can perform with sub-millisecond max pause times). We will cover it in detail in subsequent problems.

Shenandoah Garbage Collector

Shenandoah Garbage Collector was introduced in JDK 12 (and became more reliable in JDK 17) as a very low-latency, highly responsive GC (sub-millisecond pauses). It performs its job (including compaction) concurrently with the application.

Shenandoah pauses are extremely short and independent of the heap size. Garbage collecting a 1 GB heap or a 300 GB heap should produce similar pauses.

Concurrent Mark Sweep (CMS) collector (deprecated)

CMS is a *mostly concurrent* collector deprecated by G1. Since it is deprecated, I will not talk about it further.

249. Introducing G1

The G1 Garbage Collector is probably the most mature, maintained, and improved GC in Java. It was introduced in JDK 7 update 4, and from JDK 9, it became the default GC. This GC sustains high throughput and low latency (a few hundred milliseconds), being known for its balanced performance.

Internally, G1 splits the heap into equally small chunks (max size of 32 MB), which are independent of each other and can be allocated dynamically to *Eden*, *Survivor*, or *Tenured* spaces. Each such chunk is called the G1 *heap region*. So, G1 is a region-based GC.

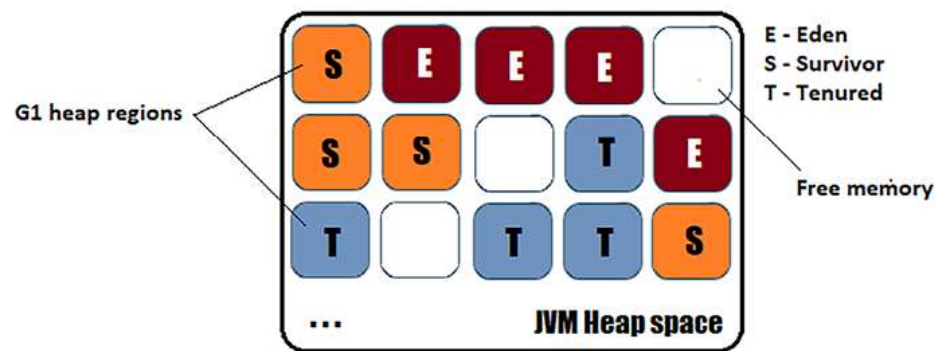


Figure 12.15: G1 splits the memory heap into equal small chunks

This architecture has a significant number of advantages. Probably, the most important one is represented by the fact that the *Old* generation can be cleaned up efficiently by cleaning it up in parts that sustain low latency.

For a heap size smaller than 4 GB, G1 will create regions of 1 MB. For heaps between 4 and 8 GB, G1 will create regions of 2 MB, and so on, up to 32 MB for a heap of 64 GB or larger. Basically, the JVM sets a number of regions that have a power of 2 and between 1 and 32 MB (typically, during the application start, the JVM sets up around 2,000+ regions).

Design principles

G1 was designed on a set of principles, as follows:

- Balanced performance – Designed to balance throughput and low latency to sustain performance.
- Generational – Dynamically split the heap into the *Young* and *Old* generations and focus on the *Young* generation, since in this region there is more garbage (most objects die in the *Young* generation region). The idea that most objects are short-lived is also known as the *generational hypothesis*.
- Incremental collecting of *Old* generation – G1 eventually moves objects from the *Young* to the *Old* generation and leaves them there to die slowly and collects them incrementally.
- Mostly concurrent – G1 strives to perform heavy tasks next to the application (concurrently) with low and predictable pauses.

Thanks to these design principles, G1 has deprecated the CMS collector.

250. Tackling G1 throughput improvements

G1 has made major progress from JDK 8 to JDK 20. Some of these improvements have been reflected in throughput. Of course, this throughput improvement is dependent on a lot of factors (application, machine, tuning, and so on) but you may expect at least 10% higher throughput in JDK 18/20 than in JDK 8.

In order to increase throughput, G1 has passed through several changes, as follows.

Delaying the start of the Old generation

Starting with JDK 9, G1 is heavily focused on collecting garbage from the *Young* generation while delaying the start (initialization, resource allocation, and so on) of the *Old* generation to the last moment (it anticipates when the *Old* generation should be started).

Focusing on easy pickings

By easy pickings, we mean objects that are short-lived (for instance, temporary buffers), occupy a significant amount of heap, and can be collected easily at low cost with important benefits. Starting with JDK 9, G1 is highly focused on easy pickings.

Improving NUMA-aware memory allocation

NUMA stands for non-uniform memory access and it was described in *Problem 245*. G1 takes advantage of NUMA from JDK 14 and it is continuously improved. If NUMA is enabled, then JVM requires the OS to place G1 heap regions on NUMA nodes. At the end of this process, the whole heap should be located evenly across the NUMA nodes that are active.

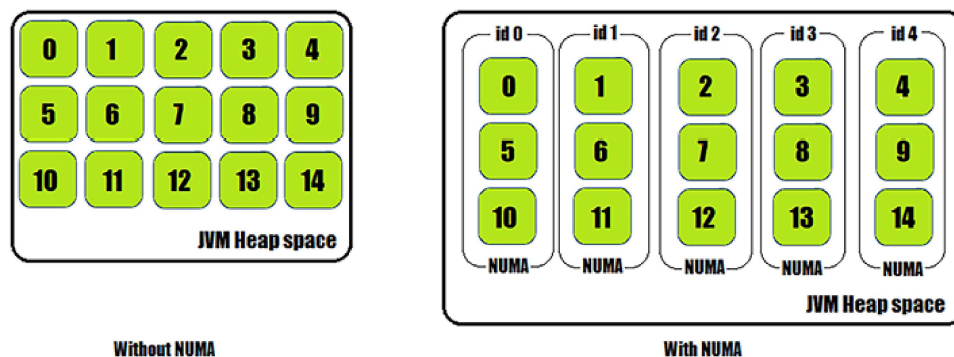


Figure 12.16: Heap memory without and with NUMA

The relationship between G1 heap regions and memory pages
(operating system pages –

[https://en.wikipedia.org/wiki/Page_\(computer_memory\)](https://en.wikipedia.org/wiki/Page_(computer_memory)).

falls into one of these two cases:

- If the size of a G1 heap region is greater than or equal to the size of a memory page, then a G1 heap region will consist of multiple memory pages (*Figure 12.17*, left-hand side).
- If the size of a G1 heap region is smaller than or equal to the size of a memory page, then a memory page will consist of multiple G1 heap regions (*Figure 12.17*, right-hand side).

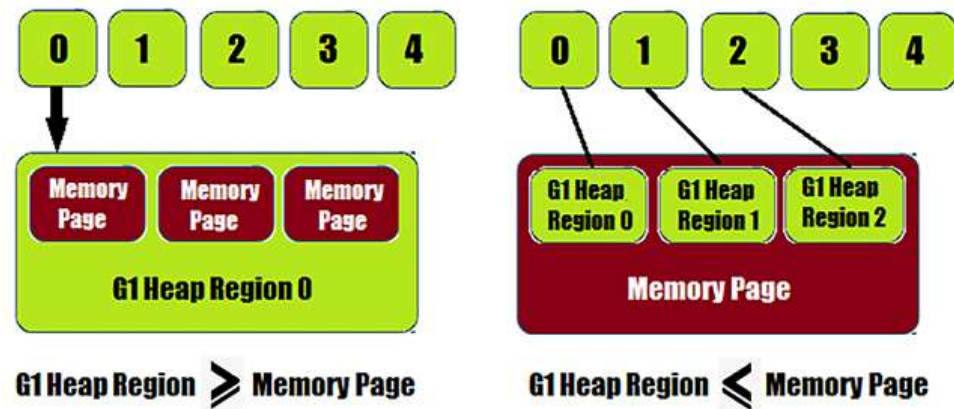


Figure 12.17: G1 heap region and memory page relationship

Without NUMA, the G1 GC allocates memory to threads from a single common memory allocator. With NUMA, there is a

memory allocator per NUMA node, and memory is allocated to threads based on these NUMA nodes.

Improving NUMA allocation awareness is a continuous goal of G1.

Parallelized full-heap collections

Among other not-so-common optimizations, we have the parallelization of full-heap collection. This was added in JDK 10 as a solution to make full-heap collections as fast as possible.

Other improvements

Tons of small improvements have been added to JVM itself, and their effects are reflected in GC performance as well. This means that by simply updating to the latest JDK, our GC will perform better. You'll notice an improvement of at least 10% between JDK 8 and JDK 20.

251. Tackling G1 latency improvements

G1 GC latency has also recorded some improvements from JDK 8 to JDK 20 (which are obviously reflected in G1 GC throughput as well).

In order to decrease latency, G1 has passed through several changes, as follows.

Merge parallel phases into a larger one

Starting with JDK 8, many aspects of G1 have been parallelized. In other words, at any moment in time, we may have in execution multiple parallel phases. Starting with JDK 9, these parallel phases can be merged into a single larger one. In practice, this means less synchronization and less time spent creating/destroying threads. As a result, this improvement speeds up the parallelization processing, leading to less latency.

Reduction of metadata

Reduction of metadata was added in JDK 11. Practically, G1 attempts to manage less metadata by reducing its amount as much as possible. Less data to manage means better latency. Of course, this means a smaller footprint as well.

Better work balancing

Work balancing was improved starting with JDK 11. In a nutshell, this means that threads that have finished their current work can steal work from other threads. In practice, this

means that the tasks are done faster since all threads are working (on their own work or on stolen work) and none of them are just hanging on. So, smarter algorithms have been developed to orchestrate and keep threads busy in order to finish the tasks faster and decrease latency. However, reducing the overhead of stealing work is still a subject of improvement.

Better parallelization

Better parallelization is available starting with JDK 14. In practice, G1 removes all duplicates from potential areas with references. Afterward, it applies parallelization instead of brute force.

Better reference scanning

In order to sustain better parallelization, JDK 15 has also improved the reference scanning in the collected areas. JDK 14 knows how to remove duplicates and parallelize the data processing, while JDK 15 knows how to scan references more optimally. Their effects are combined into decreasing latency.

Other improvements

A lot of time has been spent improving so-called uncommon situations. For instance, special attention has been focused on

evacuation failures (the attempt of moving the *live objects* between two memory areas and compacting them is known as *evacuation fashion*, and when moving objects around leads to out-of-memory issues, then we have an *evacuation failure*).

This corner case has been seriously improved in order to handle such scenarios faster than before (before JDK 17).

252. Tackling G1 footprint improvements

Between JDK 8 and JDK 20, the G1 footprint has been improved by focusing on efficient metadata and freeing the memory as quickly as possible.

In order to optimize its footprint, G1 has passed through several changes, as follows.

Maintain only the needed metadata

In order to maintain only the needed metadata, JDK 11 is capable of concurrently (re)creating the needed data and freeing it as fast as possible. In JDK 17, the focus on the needed metadata has been reiterated and only the absolutely required data is kept around. Moreover, JDK 18 comes up with a denser representation of data. All these improvements are reflected in a smaller footprint.

Release memory

Starting with JDK 17, the G1 GC is capable of concurrently releasing memory (giving it back to the OS). This means that memory can be optimally reused and is available to serve other tasks.

253. Introducing ZGC

Z Garbage Collector (ZGC) was introduced for the first time (as an experimental feature) in JDK 11. It was promoted to the production stage (production ready) in JDK 15 under JEP 377. It continues to be improved as we speak – in JDK 21, ZGC sustains application performance by maintaining separate generations for young and old objects. Basically, this minimizes allocation stalls and heap memory overhead. Moreover, JDK 21 (JEP 439) has promoted ZGC's status from Targeted to Completed.

ZGC is concurrent (works at the same time as the application based on low-level concurrency primitives such as *load barriers* and *colored pointers*), tracing (traversing the object graph to identify *live* and *non-live* objects), and compacting (fight against fragmentation). It is also NUMA-aware and region-based.

ZGC was specially designed as a low-latency, highly scalable GC capable of handling from small (a few megabytes; the docu-

mentation states 8 MB) to massive heaps (terabytes; the documentation states 16 TB) with pauses (pulse times) of a maximum of a few milliseconds (the documentation states sub-millisecond max pause times).

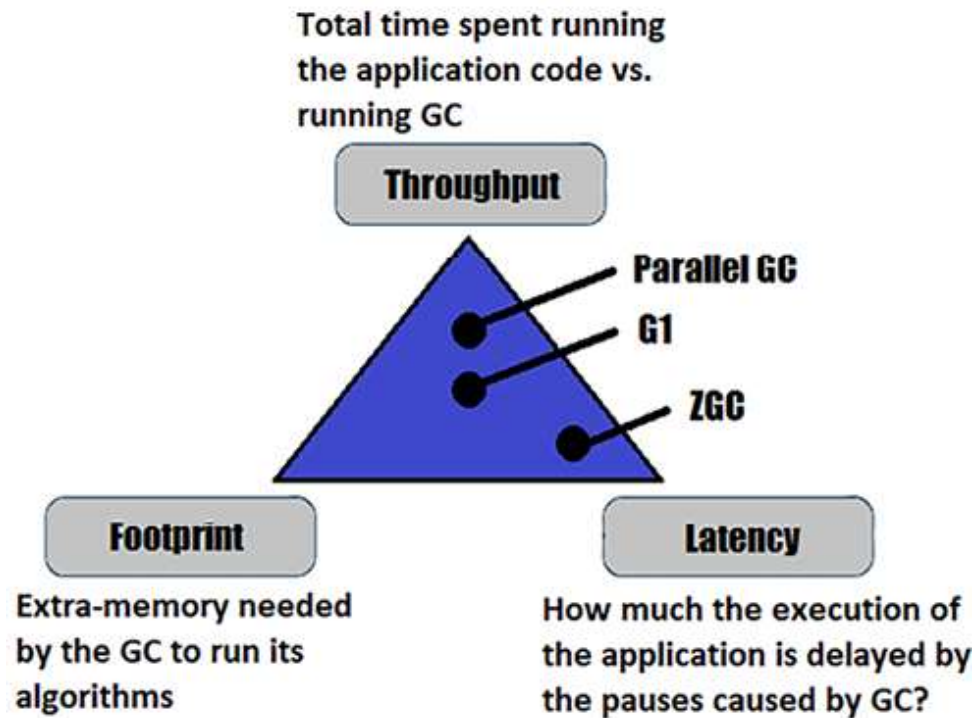


Figure 12.18: ZGC is focused on low latency

It is very important to say that pauses don't increase with the heap size (pulse times are $O(1)$, so they execute in constant time).

The shortcoming (trade-off) of ZGC relies on throughput. In other words, ZGC throughput is slightly reduced (a few percent, for instance from 0% to 10% in some cases) in comparison with G1 throughput.

Starting with JDK 16, ZGC takes advantage of a concurrent thread stack, and starting with JDK 18, it supports string de-duplication. These are just two of the major improvements next to many other improvements.

ZGC is auto-tuned. In other words, as you'll see in *Problem 256*, ZGC has just a few options that we can tune while most of the tuning is automatic.

ZGC is concurrent

G1 and ZGC are both concurrent, but they don't follow the same path. ZGC strives to collect as much garbage as possible in a concurrent fashion. For this, ZGC relies on three main lightweight (very short, sub-millisecond) pauses and three concurrent phases, as in the following figure:

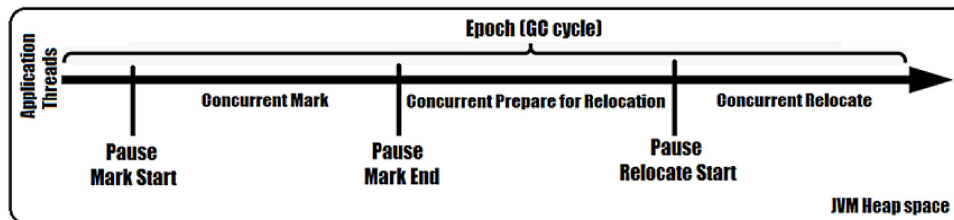


Figure 12.19: ZGC concurrency

Each of the three phases is signaled by a synchronization point (pause):

- *Pause Mark Start* – This pause signals that the *Concurrent Mark* phase follows. During this synchronization point, ZGC prepares the current state for executing the *Concurrent Mark* phase. This is a lightweight pause that performs some settings on *colored pointers* and resets a few flags and counters. Next, the *Concurrent Mark* phase runs concurrently and marks the objects from the heap.
- *Pause Mark End* – This pause signals the end of the *Concurrent Mark* phase. It also pauses before executing the *Concurrent Prepare for Relocation* phase. This phase is responsible for locating all the *live* objects from sparsely populated regions and marks them as candidates to be moved/evacuated to other regions. Moreover, during this phase, ZGC deallocates regions that don't contain *live* objects.
- *Pause Relocate Start* – This pause signals that the *Concurrent Relocate* phase follows. During this phase, the objects marked as evacuation candidates in the previous phase are effectively moved (copied) from the current region to the new region. Their references are also restored,

being deallocated from the current region and reallocated to the new region.

ZGC and colored pointers

ZGC runs next to the application and manipulates (moves around) the objects used by that application. This may lead to unexpected errors (for instance, the application may attempt to use out-of-date references) that cause the application to act weirdly or even crash. In order to prevent such scenarios, ZGC relies on two low-level concurrency primitives known as *colored pointer* and *load barriers*.

A *colored pointer* is a 64-bit pointer. This pointer is used by ZGC with a 44-bit object address capable of handling up to 16 terabytes.

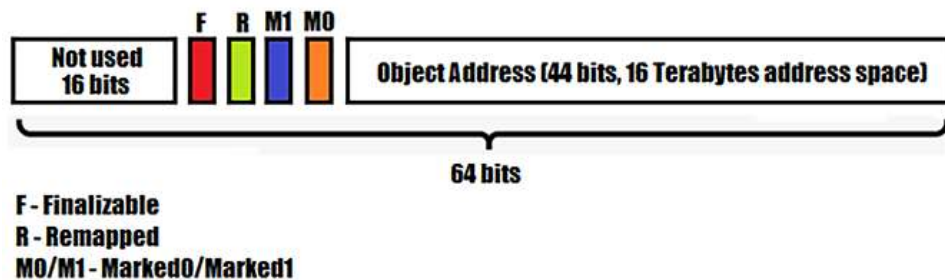


Figure 12.20: Colored pointer

A *colored pointer* reserves 20 bits for storing metadata about this pointer. The most important metadata is:

- *Finalizable* – 1 bit that indicates if an object is reachable (live object)
- *Remapped* – 1 bit that indicates if an object doesn't point into a relocation set
- *Marked0/Marked1* – 2 bits that indicate if an object is or is not marked

As well as colored pointers, ZGC needs load barriers.

ZGC and load barriers

Load barriers are portions of code injected by the compiler to handle *colored pointers*. While the application is not aware of *colored pointers*, ZGC needs to interpret and work with them, and this is exactly the job of *load barriers*. For instance, let's assume that we have, in the application, the following snippet of code (I've intentionally added the line numbers manually):

```
1: Manager manager = company.manager;  
2: Manager cManager = manager;  
3: manager.attendMeeting();  
4: int employeeNr = company.size;
```

The compiler analyzes the code to decide where to inject a *load barrier*. The conclusion is that the only place where a *load barrier* should be injected is between lines 1 and 2 because that is the only place where an object is loaded from the heap. In line 2, there is no need for a *load barrier* since there is a copy of the memory reference. In line 3, there is also no need for a *load barrier* since there is a method reference. Finally, in line 4, there is no need for a *load barrier* since there is no object reference. So, ZGC sees this code as follows:

```
1: Manager manager = company.manager;  
   <load barrier injected at this point>  
2: Manager cManager = manager;    // copying reference  
3: manager.attendMeeting();        // method reference  
4: int employeeNr = company.size;  // no object reference
```

The purpose of the *load barrier* is to ensure that the pointers are valid (shown by having a good color). If a bad color is encountered, then the *load barrier* tries to heal it (update the pointer, relocate the object reference, and so on).

ZGC is region-based

ZGC is a *region-based* GC, so it divides the heap into smaller regions/chunks that are allocated to the *Young* or *Old* generation, as in the following figure:

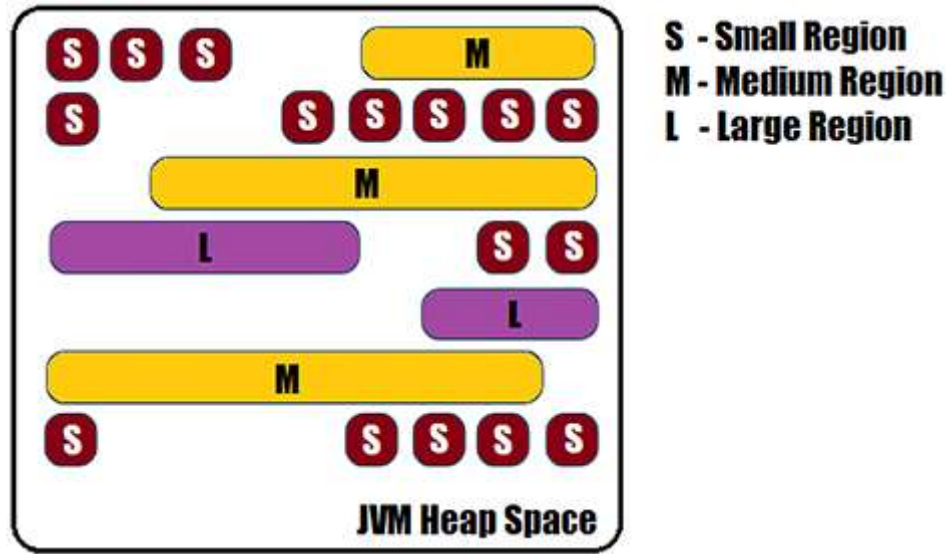


Figure 12.21: ZGC heap regions

Exactly like G1, ZGC is a *region-based* GC. Nevertheless, ZGC is more powerful than G1 and has the capability to dynamically increase/decrease the number of active regions during run-time. Moreover, ZGC can rely on regions of three sizes, as follows:

- Small region – These regions are 2 MB.
- Medium region – These regions can be from 4 MB to 32 MB. They are dynamically sized.
- Large region – These are regions reserved for *humongous* objects. These are tightly fitted regions that can be smaller or larger than a medium region.

So, at a glance, ZGC is a concurrent GC with constant pause times (sub-millisecond), working in parallel mode, and capable of fighting against fragmentation via compacting. Moreover, it is region-based, NUMA-aware, capable of auto-tuning, and relies on *colored pointers* and *load barriers*.

254. Monitoring garbage collectors

Monitoring the activity and evolution in the timeline of your GC is a major aspect in order to identify potential performance issues. For instance, you may be interested in monitoring pause times, identifying the frequency and types of GC events, what spaces are filled up by the triggered GC events, and so on. The main goal is to collect as much information as possible that can be helpful in troubleshooting performance issues related to heap memory and GC evolution.

Any modern IDE provides profilers that contain (among other related things) information and real-time graphs about the GC *epochs/cycles*. For instance, the following figure is from the NetBeans IDE, which displays the GC evolution (heap status) as an item of the toolbar (by simply clicking on that area, you can force the GC to perform garbage collection):



Figure 12.22: NetBeans display GC evolution on the toolbar

Of course, a more detailed view is available via the NetBeans profiler:

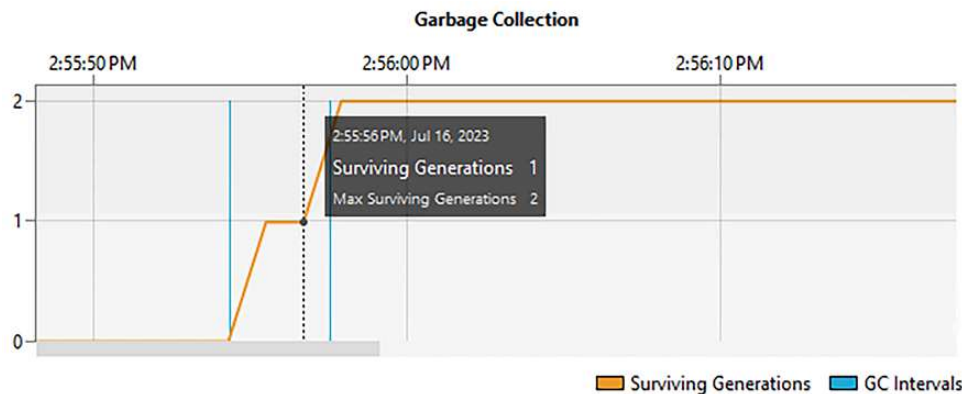


Figure 12.23: NetBeans profiler for GC

Among other tools that can be used to monitor your GC are the *jstat* command-line utility (`jstat -gc $JAVA_PID`) and *JConsole* (Java Monitoring and Management Console).

The following figure is a screenshot from *JConsole*:

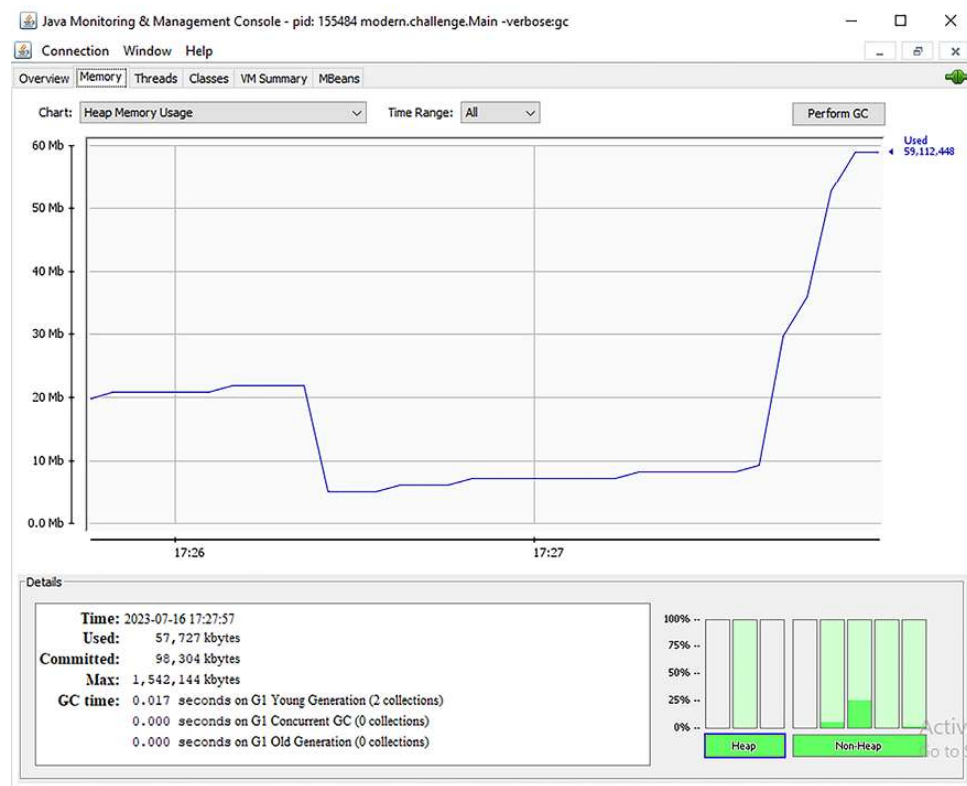


Figure 12.24: Monitoring the GC via JConsole

You may also be interested in **visualgc** (Visual Garbage Collection Monitoring Tool) from Oracle, **JDK VisualGC** (IntelliJ IDE plugin), and **Memory Analyzer (MAT)** from Eclipse.

255. Logging garbage collectors

Analyzing the GC logs is another approach that can be useful for finding memory issues. Since GC logs don't add a signifi-

cant overhead, they can be enabled in production for debugging purposes. Really, GC logs have an insignificant overhead, so you should definitely use them!

Let's consider some simple Java code that adds and removes from `List<String>`. Adding and removing the code requires a full GC via `System.gc()`:

```
private static final List<String> strings = new ArrayList<>();
...
logger.info("Application started ...");
String string = "prefixedString_";
// Add in heap 5 millions String instances
for (int i = 0; i < 5_000_000; i++) {
    String newString = string + i;
    strings.add(newString);
}
logger.info(() -> "List size: " + strings.size());
// Force GC execution
System.gc();
// Remove 10_000 out of 5 millions
for (int i = 0; i < 10_000; i++) {
    String newString = string + i;
    strings.remove(newString);
}
logger.info(() -> "List size: " + strings.size());
logger.info("Application done ...");
```

Next, we want to run this simple application and log the GC activity.

Before JDK 9, we can obtain a quick and verbose log of GC via the `-verbose:gc` option:

```
java ... -verbose:gc
```

A possible output will look as follows:

```
[0.319s][info][gc] Using G1
[17:03:47] [INFO] Application started ...
[0.917s][info][gc] GC(0) Pause Young (Normal)
    (G1 Evacuation Pause) 27M->24M(96M) 34.391ms
[0.948s][info][gc] GC(1) Pause Young (Normal)
    (G1 Evacuation Pause) 40M->40M(96M) 21.300ms
[0.986s][info][gc] GC(2) Pause Young (Normal)
    (G1 Evacuation Pause) 60M->60M(96M) 24.085ms
[0.997s][info][gc] GC(3) Pause Young (Concurrent Start)
    (G1 Humongous Allocation) 63M->64M(96M) 8.072ms
[0.997s][info][gc] GC(4) Concurrent Mark Cycle
[1.030s][info][gc] GC(5) Pause Young (Normal)
    (G1 Evacuation Pause) 78M->78M(288M) 17.036ms
[1.059s][info][gc] GC(4) Pause Remark 101M->94M(288M) 0.867ms
[1.083s][info][gc] GC(4) Pause Cleanup 109M->109M(288M) 0.14ms
[1.085s][info][gc] GC(4) Concurrent Mark Cycle 87.261ms
[1.125s][info][gc] GC(6) Pause Young (Prepare Mixed)
    (G1 Evacuation Pause) 116M->118M(288M) 32.640ms
```

```

[1.220s][info][gc] GC(7) Pause Young (Mixed)
      (G1 Evacuation Pause) 181M->181M(288M) 42.497ms
[1.257s][info][gc] GC(8) Pause Young (Concurrent Start)
      (G1 Humongous Allocation) 200M->201M(288M) 23.297ms
[1.257s][info][gc] GC(9) Concurrent Mark Cycle
[1.316s][info][gc] GC(10) Pause Young (Normal)
      (G1 Evacuation Pause) 243M->244M(288M) 24.492ms
[1.345s][info][gc] GC(11) Pause Young (Normal)
      (G1 Evacuation Pause) 256M->258M(776M) 12.445ms
[1.400s][info][gc] GC(9) Pause Remark 290M->274M(776M) 0.732ms
[1.461s][info][gc] GC(9) Pause Cleanup 335M->335M(776M) 0.25ms
[1.466s][info][gc] GC(9) Concurrent Mark Cycle 209.289ms
[1.531s][info][gc] GC(12) Pause Young (Prepare Mixed)
      (G1 Evacuation Pause) 344M->345M(776M) 54.939ms
[17:03:48] [INFO] List size: 5000000
[1.830s][info][gc] GC(13) Pause Full (System.gc())
      368M->330M(776M) 277.793ms
[17:04:15] [INFO] List size: 4990000
[17:04:15] [INFO] Application done ...

```

This is the simplest GC log. For more details, we can add the `-XX:+PrintGCDetails` option:

```
java ... -XX:+PrintGCDetails -verbose:gc
```

Moreover, we can attach a few options for obtaining information about tenuring distribution (`-XX:+PrintTenuringDistribution`), garbage collector time

stamps (`-XX:+PrintGCTimeStamps`), the class histogram (`-XX:+PrintClassHistogram`), and the application stopped time (`-XX:+PrintGCApplicationStoppedTime`).

In this context, GC logs are available on the console (stdout) using the info level. You can easily redirect the GC logs to a file via the `-Xloggc` option:

```
java ... -verbose:gc -Xloggc:gclog.txt
```

Actually, `-Xloggc` is deprecated and you should use it only if you are using a JDK earlier than version 9. Starting with JDK 9 (JEP 158 – <https://openjdk.org/jeps/158>), we have a *unified logging system* for all JVM components.

So, starting with JDK 9, we have a unified logging system via the `-Xlog` option. The equivalent of `-XX:+PrintGCDetails -verbose:gc` is `-Xlog:gc*`. If we want to redirect GC logs to a file using the debug level, then we can do it as follows:

```
java ... -Xlog:gc*=debug:file=gclog.txt
```

The `gclog.txt` will be saved in the application root folder. If you remove the `*` character, then you'll get a less verbose GC log.

Logging only NUMA logs is available via `-Xlog:numa*={log level}`.

Having the GC log is half of the problem. The other half consists of interpreting this log. As you can see, this is not that easy. Fortunately, you don't have to bother reading the log files because we have tools capable of parsing, analyzing, and providing detailed reports from GC logs.

One of these tools is Universal GC Log Analyzer (<https://gceasy.io/>). Using the free version, we can upload our `gclog.txt` file and get a detailed report. For instance, in the following figure, we can see how memory was allocated for our application.

Generation	Allocated 	Peak 
Young Generation	119 mb	81 mb
Old Generation	657 mb	285 mb
Humongous	n/a	51 mb
Meta Space	1.12 mb	950 kb
Young + Old + Meta space	777.12 mb	369.93 mb

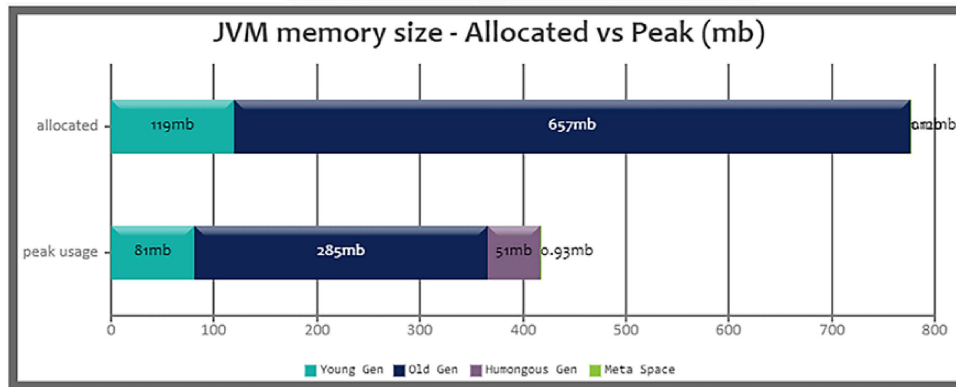


Figure 12.25: A screenshot from Universal GC Log Analyzer (GCEasy) report

This figure is just a very small part of the report. Try it yourself to see the full report. Other similar tools that you may like to try are GCViewer, GCPlot, IBM Garbage Collection and Memory Visualizer, garbagecat, SolarWinds Loggly, Sematext Logs, **Java Flight Recorder (JFR)**, jvm-gc-logs-analyzer, and so on.

256. Tuning garbage collectors

Garbage collectors are complex machinery whose performances are highly related to their settings (startup parameters) in the context of the current JVM, current application, and hardware. Since the GC consumes and shares resources (memory, CPU time, and so on) with our application, it is essential to tune it to work as efficiently as possible. If the GC is not efficient, then we may face significant pause times that will negatively impact the application run.

In this problem, we will cover the main tuning options available for the serial GC, parallel GC, G1 GC, and ZGC.

How to tune

Before attempting to tune the GC, ensure that it is really causing trouble. By inspecting and correlating the charts and logs, you can identify such troubles and decide where you should act (what parameters should be tuned). Check out the usage of the heap memory and how objects fill up the *Eden*, *Survivor*, and *Tenured* spaces.

Typically, a healthy GC produces a heap usage graph known as *shark teeth*, as in the following figure:



Figure 12.26: Healthy heap usage

Moreover, check out the 90th and 99th percentiles along with the average GC time. This information can give you a hint about whether more memory is needed or if it is cleared properly.

Once you identify the GC troubles, try to tackle them one by one. Don't rush to change several parameters at once because it will be hard to manage and analyze their combined effect. Try to modify one of them and experiment to see what's happening and what the results are. If you see some benefits, then go for the next one and experiment again. Observe if the combined effect has been improved or not. Otherwise, maybe it is better to restore this one to its default value before going for the next one.

Tuning the serial garbage collector

The serial garbage collector can be enabled via `-XX:+UseSerialGC`.

Since this is a single-threaded GC, there is not much to tune. However, you may adjust the heap size via `-Xmx` and `-Xms` (for instance, a heap size of 3 GB can be set via `-Xmx3g` and `-Xms3g`) and the *Young* generation size via the `-Xmn` option.

Nevertheless, these options work with all types of GC for setting the heap size.

Tunning the parallel garbage collector

The parallel garbage collector can be enabled via `-XX:+UseParallelGC`.

This GC is multi-threaded and we can control the number of threads used for cleaning tasks via the `-XX:ParallelGCThreads` option (for instance, setting six threads can be done with `-XX:ParallelGCThreads=6`).

Keep in mind that a higher number of threads results in a higher fragmentation of the heap reserved for the *Tenured* space. Each thread that participates in a *Minor* GC event will reserve some space in the *Tenured* space for its promotions goals. This will lead to serious fragmentation of the *Tenured* space. Fixing this issue requires reducing the number of threads and increasing the size of the *Old* generation.

The maximum pause time can be controlled via the `-XX:MaxGCPauseMillis` option (for instance, `-XX:MaxGCPauseMillis=150`, which will ensure maximum pause times of 150 milliseconds between two consecutive runs/events of GC). However, be aware that bigger pause times will allow more garbage to hit the heap. This means that the

next run of the GC will be more expensive. On the other hand, a small pause time will instruct the GC to run more frequently, and this may cause the application to spend too much time on garbage collection.

Next, the maximum throughput that we want to achieve can be set via the `-XX:GCTimeRatio` option. This option is evaluated as the ratio between the time spent inside vs. outside the GC. It is a percentage computed as $1/(1 + n)$. In other words, `-XX:GCTimeRatio` specifies the amount of time dedicated to garbage collection in a $1/(1+n)$ ratio.

For instance, if we set this option as `-XX:GCTimeRatio=14`, then we target a goal of $1/15$. This means that 6% of the total time should be spent in garbage collection (by default, this option is set to 99, or 1% of time is spent on garbage collection).

If you get an `OutOfMemoryError`, then most probably, this is caused by too much time spent on garbage collection. For instance, if more than 98% of the time is spent recovering less than 2% of the heap, then you'll see such an error. In other words, GC has spent a lot of time cleaning a small part of the heap. This may indicate memory leaks or a heap that is too small. Nevertheless, if you can live with this error, then you can suppress it via the `-XX:-UseGCOverheadLimit` option.

We can also control the size of the *Young/Old* generation. You can control the growth of the *Young* generation via `-XX:YoungGenerationSizeIncrement`, and the growth of the *Old* generation via `-XX:TenuredGenerationSizeIncrement`. The values of these options are percentages (by default, the growth percentage is 20% and the shrinking percentage is 5%). Moreover, you can control the shrinking percentage by simply setting the `-XX:AdaptiveSizeDecrementScaleFactor` option. The shrinking of the *Young* generation is automatically computed via $\frac{-XX:YoungGenerationSizeIncrement}{-XX:AdaptiveSizeDecrementScaleFactor}$.

Tuning the G1 garbage collector

The G1 garbage collector can be enabled via `-XX:+UseG1GC`.

By default, G1 takes care of the *Young* generation. Basically, it cleans the *Young* generation and promotes the reachable objects to the *Old* generation until it hits a threshold of 45%. This default value can be altered via `-XX:InitiatingHeapOccupancyPercent`.

When tuning the G1 collector, we can target throughput, latency, or footprint. When tuning for latency, we have to focus on low pause times. This can be achieved by setting the `-Xmx` and `-Xms` options at the same value (to avoid heap resizing).

Moreover, we can rely on the `-XX:+AlwaysPreTouch` and `-XX:+UseLargePages` flag options to load the (large) memory pages at the start of the application.

If latency is affected by the *Young* generation size, then it is a good idea to decrease its size via `-XX:G1NewSizePercent` and `-XX:G1MaxNewSizePercent`. On the other hand, if the *Mixed* GC events affect latency, then we should focus on spreading the *Tenured* space across more collections via the `-XX:G1MixedGCCountTarget` flag option. In addition, we may want to focus on `-XX:G1HeapWastePercent` (stop earlier the *Tenured* space cleanup) and `-XX:G1MixedGCLiveThresholdPercent` (the *Tenured* space becomes part of a mixed collection only when this threshold is exceeded (defaults to 65)). You may also be interested in `-XX:G1RSetUpdatingPauseTimePercent`, `-XX:-ReduceInitialCardMarks`, and `-XX:G1RSetRegionEntries` (for details, see the G1 documentation).

When tuning for throughput (applications that manipulate a lot of data need a GC capable of cleaning as much garbage as possible), we have to focus on the `-XX:MaxGCPauseMillis` option. When this option has a low effect, then you should focus on `-XX:G1NewSizePercent` and `-XX:G1MaxNewSizePercent`. Basically, G1 strives to bound the *Young* generation size between the values of `-XX:G1NewSizePercent` (default is 5) and

`-XX:G1MaxNewSizePercent` (default is 60). By juggling these three options, we can relax the GC and give it more time and space to process a lot of garbage. In addition, throughput can be sustained via the `-XX:G1RSetUpdatingPauseTimePercent` option.

By increasing the value of this option, we decrease the time spent in concurrent parts while performing more work when pausing the application's threads. In addition, as in the case of tuning for latency, we may want to avoid heap resizing (set `-Xmx` and `-Xms` at the same value) and turn on the `-XX:+AlwaysPreTouch` and `-XX:+UseLargePages` flag options.

Tuning for footprint may be influenced by setting the `-XX:GCTimeRatio`. This defaults to 12 (8%) but we can increase it to force the GC to spend more time in garbage collection. As a result, more heap memory will be free, but this is not a general rule. It is recommended to experiment and see how it really works. Moreover, since JDK 8 (update 20), we can set up the `-XX:+UseStringDeduplication` flag option. Practically, if this option is enabled, then G1 locates duplicate strings and holds a single reference to one string while cleaning up the duplicates. This should result in a more efficient and optimal usage of the heap memory. You also may want to consult the documentation for `-XX:+PrintStringDeduplicationStatistics` and `-XX:StringDeduplicationAgeThreshold=n`.

As you already know, G1 splits the heap into small regions up to 32 MB. In practice, this may lead to performance degradation, especially for large objects on very large heaps. But, starting with JDK 18, the maximum region size was set up to 512 MB. Whenever you need, you can control the maximum region size via the `-XX:G1HeapRegionSize`.

Tuning Z Garbage Collector

Z Garbage Collector can be enabled via `-XX:+UseZGC` (before JDK 15, you may also need `-XX:+UnlockExperimentalVMOptions`).

One of the most important settings of this GC is `-Xmx` to set up the maximum heap size. Next to this one, we have `-XX:ConcGCThreads=n`, where `n` is the number of threads used by ZGC. However, ZGC is fully capable of dynamically determining the optimal value for this option, so think twice before modifying it.

Tuning Metaspace (Metadata space)

If your focus is on tuning Metaspace, then you'll be interested in the following options:

- `-XX:MetaspaceSize` – Set the initial size of Metaspace

- `-XX:MaxMetaspaceSize` – Set the maximum size of Metaspace
- `-XX:MinMetaspaceFreeRatio` – Set the class metadata capacity that should be free after running the GC (this is the minimum value as a percentage)
- `-XX:MaxMetaspaceFreeRatio` – Set the class metadata capacity that should be free after running the GC (this is the maximum value as a percentage)

Controlling the size and behavior of the Metaspace can also be part of tuning your GC. Again, experimenting and comparing the results is the main rule of thumb that can lead you to a successful and optimal GC.

257. Introducing Application Class Data Sharing (AppCDS, or Java's Startup Booster)

Launching a Java application is a multi-step process. Before executing the bytecode of a class, the JVM has to perform at least the following steps for a given class name:

1. Look up the class on disk (JVM has to scan the disk and find the given class name).
2. Load the class (JVM opens the file and loads its content).

3. Check the bytecode (JVM verifies the integrity of the content).
4. Pull the bytecode internally (JVM transfers the code into an internal data structure).

Obviously, these steps are not cost-free. Loading hundreds/thousands of classes will have a significant overhead on launching time and memory footprint. Typically, an application's JAR remains unchanged for a long time, but JVM performs the previous steps and obtains the same result every time we launch the application.

Improving/accelerating the startup performance and even reducing the memory footprint are the main goals of Application Class Data Sharing (AppCDS). In a nutshell, AppCDS was initially popularized in JDK 10 (2018), and it was simplified in JDK 13 and JDK 19. The idea of AppCDS is to perform the previous steps once and dump the result into an archive. This archive can be reused for subsequent launches and even shared across multiple JVM instances running on the same host. The bigger the application is, the bigger the startup benefits are.

Putting these ideas into practice requires the following three steps:

1. Create the list of classes that should be shared between the application instances.

2. Archive this list of classes in an archive suitable for memory mapping.
3. Give the resulting archive to every application startup (every application instance).

Depending on the JDK used, you may have to manually follow these steps or only a part of them. The AppCDS algorithm is constantly improved, so its use depends on your JDK, as follows:

- In JDK 10/11, you have to follow the previous three steps. However, if you want to share only the JDK classes (not the application classes), then you can skip step 1. JDK has already prepared the list of classes that should be shared in `$JAVA_HOME\lib\classlist` (there are around 1,200 classes).
- In JDK 12+, you can skip steps 1 and 2 because an archive of JDK classes is already available. However, if you want to share the application classes as well, then you need to follow all three steps.
- In JDK 13+, we can take advantage of dynamic CDS archives. Practically, JVM collects the classes to be added into the archive at application runtime. Steps 1 and 2 are merged automatically.
- In JDK 19+, we can take advantage of the autogenerated shared archive. The CDS archive is built and used in a single

command.

Tackling a JDK class data archive

Tackling a JDK class data archive means that we will create a reusable archive containing only JDK classes, not the classes of our application.

JDK 10/JDK 11

In JDK 10/11, we can use the already existent `$JAVA_HOME/lib/classlist`. This is a file that contains the list of JDK classes, and you can easily inspect it with a text editor. Having the class list, we can create the proper CDS archive via the `-Xshare:dump` option, as follows:

```
java ... -Xshare:dump
```

The resulting archive will be stored in `$JAVA_HOME\bin\server\classes.jsa` (this is the default location and you may need to run this command as an administrator to avoid a permission denied restriction).

Next, we can use this archive via `-Xshare:on`, as follows (if you run under JDK 11, then `--enable-preview` is also needed, but I'll skip it here):

```
java ... -Xshare:on
```

We can use the unified logging system to track CDS work via `-Xlog`, as follows:

```
java ... -Xshare:on -Xlog:class+load:file=cds.log
```

In the output, we can see that a shared object is marked with a significant message, as follows (objects that are not shared don't contain the “*shared objects file*” text):

```
[6.376s][info][class,load] java.lang.Object source: shared objects file
```

By default, JVM scans for archives in the default location. But, if we move the archive to another location (for instance, in the application root folder), then we have to indicate this location via `-XX:SharedArchiveFile`, as follows:

```
java ... -Xshare:on -XX:SharedArchiveFile=./classes.jsa  
-Xlog:class+load:file=cds.log
```

Actually, the default value of `-Xshare` is `auto`. This means that if an archive is found, then it is used automatically. So, if you omit `-Xshare:on`, then JVM relies on `-Xshare:auto`,

which has the same effect. If you want to shut down CDS support, then use `-Xshare:off`.

JDK 12+

JDK 12+ comes with an already prepared archive for JDK classes, so there is no need to create one (no need to use `-Xshare:dump`). JVM will use it automatically thanks to `-Xshare:auto` or via the explicit `-Xshare:on`:

```
java ... -Xshare:on -Xlog:class+load:file=cds.log
```

Of course, if you have a different location than the default one (`$JAVA_HOME\bin\server\classes.jsa`), then use `-XX:SharedArchiveFile`.

Tackling application class data archive

Besides the JDK classes, we may want to share our application classes as well. This can be done in several steps depending on the JDK.

Before JDK 13

Before JDK 13, we need to create the list of classes that we want to share. We can do it manually or via the `-`

`XX:DumpLoadedClassList` option, as follows (I'll skip it here, but you'll need `--enable-preview` as well):

```
java ... -XX:DumpLoadedClassList=classes.lst
```

The generated `classes.lst` contains all classes (classes used by the JDK + your application classes) that will be shared. Next, we can obtain the archive, as follows:

```
java ... -Xshare:dump -XX:SharedClassListFile=classes.lst  
        -XX:SharedArchiveFile=appcds.jsa --class-path app.jar
```

Consider the following important note.

Important note



Notice that CDS (AppCDS) can archive classes from JAR files only. Don't use classpaths with wildcards or exploded paths such as `target/classes`. Replace `app.jar` with your JAR.

The archive (`appcds.jsa`) is stored in the application root folder, not in `$JAVA_HOME`.

Finally, we can share the archive and get some logs, as follows:

```
java ... -Xshare:on -XX:SharedArchiveFile=./appcds.jsa  
-Xlog:class+load:file=cds.log
```

Done!

JDK 13+

Starting with JDK 13, we can take advantage of *dynamic application class-data sharing*. In other words, we can obtain the archive when JVM exits via the `-XX:ArchiveClassesAtExit` option, as follows:

```
java ... -XX:ArchiveClassesAtExit=appcds.jsa -jar app.jar
```

Replace `app.jar` with your JAR. Use the generated archive as usual via `-Xshare:on` and `-XX:SharedArchiveFile`.

JDK 19+

Starting with JDK 19, we can rely on autogenerated archives. This is possible in a single command, as follows:

```
java ... -XX:+AutoCreateSharedArchive  
-XX:SharedArchiveFile=appcds.jsa -jar app.jar
```


This time, JVM checks if an archive exists at the path given via `-XX:SharedArchiveFile`. If such an archive exists, then JVM loads and uses it; otherwise, at exit, JVM will generate an archive at that location. Moreover, JVM checks the JDK version used for creating the archive. If the current JDK version (the JVM JDK version) and the archive JDK are not the same, then JVM will overwrite the existing archive.

You may also be interested in the following article:

<https://spring.io/blog/2023/12/04/cds-with-spring-framework-6-1>.

Summary

This chapter covered 15 problems with garbage collectors and AppCDS. Even if these problems have been mostly theoretical, they still represent major topics that can boost your application performance at runtime (in the GC case) and startup (in the AppCDS case).

Join our community on Discord

Join our community's Discord space for discussions with the author and other readers:

<https://discord.gg/8mgytp5DGQ>

